

P1108R4

web_view

Published Proposal, 2019-10-07

This version:

<http://wg21.link/p1108r4>

Author:

[Hal Finkel](#) (Argonne National Laboratory)

Audience:

SG1, SG12, SG13, SG16, SG18

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Abstract

This paper proposes a `web_view` facility for the C++ standard library. This facility enables modern, natural, multimodal user interaction by leveraging existing web standards and technologies.

Table of Contents

- 1 Change Log**
- 2 Introduction**
- 3 Some History and Graphics**
- 4 FAQ**
 - 4.1 What if we do nothing?
 - 4.2 Is this API surface small- and long-lived enough to be useful?
 - 4.3 Does this proposal require that standard-library implementations ship/maintain a web browser?
 - 4.4 Why not standardize an HTTP-server library instead of this?

- 4.5 Should there be some integration with WebSockets?
- 4.6 Should there be some use of executors in this library?
- 4.7 What happens when the web_view object is destroyed?
- 4.8 Isn't dealing with text (in HTML, JSON, URIs, etc.) ugly, verbose, difficult, error prone, or similar?
- 4.9 Why don't we just standardize Qt's API?

5 Proposed Interface

6 An Example

7 Very-Preliminary Wording

8 Acknowledgments

References

Informative References

§ 1. Change Log

- r4: Based on the feedback from Cologne, and afterward, significant updates: use `u8string` instead of `string`, `set_uri_scheme_handler` -> `add_uri_scheme_handler`, remove `set_close_handler`, `run_script` -> `invoke`, `request_close` -> `close`, wait changes, native-handle types, reduced standard list.
- r3: Based on the discussions in Cologne, many updates (added links to Mozilla API, use `<format>`, etc.)
- r2: Based on the SG13 discussion in San Diego, added some comments on deployment options and process separation.
- r1: Added discussion of Qt. Added links to public discussion threads in the acknowledgments.

§ 2. Introduction

Standard C++ should enable modern, natural, multimodal user interaction in order for a facility to be broadly useful for application development. Reality is that most users do not interact with applications using a command prompt (i.e., console I/O), but rather, use some graphical user interface. The C++ standard, however, provides no useful facilities in this regard, and as a result, users either need to

make use of system-specific APIs, third-party libraries, or move to a different programming language. Moreover, these interactions require both input and output facilities, where output includes 2D graphics, 3D graphics, text rendering, dialog elements, and there are many forms of input devices (especially when accessibility-related technologies are considered). While I believe that the authors of [\[P0267R7\]](#), and others, have made a commendable effort in the 2D-graphics area, down-scoping the problem to 2D graphics fails to address the underlying requirement of enabling user interaction through modern interfaces.

Unfortunately, this committee has neither the time nor the expertise to address this problem by directly creating some sufficiently-comprehensive API. Specifically, this is the problem addressed by web standards (i.e., HTML, CSS, SVG, and so on), those are in-turn built on many efforts in graphics (and many other areas), and clearly we cannot foster a comparable effort in this space in this committee. The only feasible way forward is to reach out to the large and vibrant community tackling this issue, creating portable standards in this space, and make direct use of their efforts.

Walking this path to its logical conclusion leads to an API providing a window into which a C++ program can inject web content (along with some callbacks to provide data, and possibly, handle some other events directly).

§ 3. Some History and Graphics

After a long and interestingly-introspective graphics evening session in Rapperswil, we did not have consensus to move forward in the 2D graphics space. Nevertheless, this is an important area (and see sections 4 and 5 of [\[P0939R0\]](#)), wide interest remains, and in this paper I propose taking a different approach from those previously discussed. In particular, I propose adding a `web_view` class to the standard library. I view this as taking the path represented by [\[P1062R0\]](#) to its logical conclusion. This represents, in my view, both the best approach, and the only practical approach, we can take to enable useful graphical user interaction in standard C++. In addition, graphics is not enough. We need to enable modern, natural, multimodal user interaction in order for a facility to be broadly useful for application development.

I believe that:

1. The underlying use case, which is unfortunately not well described as "2D Graphics", is important.
2. The underlying approach of [\[P1062R0\]](#), by making use of outside standards (i.e., SVG), embraces the right philosophical approach to improving standard C++ in this regard.

This proposal was reviewed by SG13 in Cologne:

We encourage further work on this paper:

SF F N A SA 3 7 5 0 0

And was reviewed in LEWGI:

We should promise more committee time to P1108 knowing that our time is scarce and this will leave less time for other work:

SF F N A SA 4 7 5 0 1

This proposal has also been reviewed by SG12, SG1, and SG16.

§ 4. FAQ

§ 4.1. What if we do nothing?

If we do nothing then standard C++ will continue to lack a way to create applications with modern user interfaces.

That having been said, one can imagine a future in which WebAssembly standardizes some interface to the DOM (document object model) and other relevant state for WebAssembly applications running within a web-content engine, and most operating systems providing a way to launch WebAssembly applications within some otherwise-standalone web-content environment. In such a future, C++ applications, compiled to WebAssembly, might have a platform-specific, but portable, way to provide a modern user interface.

While there is a desire for WebAssembly to provide access to the DOM (and other web APIs), see [WebAssembly Future Features](#) and [WebAssembly Proposal Issue 16](#), and there already exist packages providing functionality along these lines (e.g., [asm-dom](#)), it is unclear whether this will provide a sufficient solution for the C++ ecosystem at large. For one thing, WebAssembly applications may never truly match the performance of applications compiled directly for the underlying hardware. Such applications also might not be able to access necessary system services directly.

§ 4.2. Is this API surface small- and long-lived enough to be useful?

To get some idea of what this space looks like currently, I recommend looking at:

- [The WebKit \(WebKitGTK+\) API](#)
- [The Internet Explorer IWebBrowser2 API](#)

- [The macOS/iOS WebKit API](#)
- [The Android WebView API](#)
- [The Mozilla \(Gecko\) API](#)

There are a number of concepts exposed in these APIs, including things like sandboxing parameters, which seem likely to be too dynamic to reasonably standardize. We'd be relying on the library to provide "reasonable defaults" in many cases. That having been said, as with our Unicode support in standard C++, our API surface could increase over time. To be useful, we'll need to require support for a large number external standards (i.e., [X]HTML, CSS, SVG, ECMAScript, and possibly others). Our three-year release cycle is likely sufficient to maintain a proper list of such standards, but it's still a large list, and to be clear, the transitive closure of this list is huge.

Nevertheless, surveying the current implementations has convinced me that an interface along the lines of that proposed here is appropriate for standardization, at least in the sense that, while broadly useful, using these services from a C++ application today requires difficult-to-get-right platform-specific code. Moving that burden to C++ library implementers, as a result, makes sense. In addition, it should be possible to create production applications using this facility that meet modern user expectations across many different kinds of devices and platforms.

§ 4.3. Does this proposal require that standard-library implementations ship/maintain a web browser?

I think that it's important that we think of this interface as one exposing underlying platform capabilities, not as one requiring an implementation shipped as part of the C++ standard library. Required security and functionality updates on many platforms for web-content support may be far more frequent than updates to the C++ standard library, and a `web_view` in the C++ standard library should automatically use this frequently-updated web-content software.

I don't believe that we can require all C++ implementations, not even all non-freestanding C++ implementations, to provide a web-content interface. As a result, it must be legal for an implementation to stub out the implementation of `web_view` where it cannot be reasonably supported. Nevertheless, given currently-shipping web-browser implementations, we can provide a succinct API that ties C++ into the most-vibrant standards-driven ecosystem in this space.

§ 4.4. Why not standardize an HTTP-server library instead of this?

There are three reasons why standardizing an HTTP-server library might not address the same set of use cases as this proposal.

First, it does not address how to cause the web content to be displayed to the user. An external web browser might be launched and directed to some port on localhost opened by the application, but this might not provide the same look/behaviors as a native application (e.g., the name of the web browser might be visible, the localhost URL might be visible, there might not be a robust way to detect that the user closed the interface, it might appear to the user as through two applications are running, and so on). Some of these issues can be mitigated today using certain web browsers (e.g., using Chrome with the `--app` flag), but this usage mode for a web browser, which only addresses some of these concerns, is not universally supported today.

Second, it might not be possible to create a robust, secure, high-performance library without exposing protocol-level details on the interface, and doing that might create a library interface with too short of a useful lifetime. For one thing, the HTTP landscape is evolving relatively quickly: arguably, such a library should support HTTP/2 (SPDY), or even HTTP/3 (QUIC), and should certainly support WebSockets ([\[rfc6455\]](#)) (which is a layered on top of HTTP). TLS support seems like a necessary feature, which immediately implies exposing functionality related to certificate management and validation. Handling of content negotiation, cookies, and many other facets of the underlying protocol would likely need to be exposed as well. It might not be possible for us to standardize an interface here that exposes a sufficient amount of detail to constitute a high-quality implementation and yet remain useful over an extended period as the technology changes.

Third, providing an HTTP server is stepping away from the benefits of wrapping a system service, as this proposal does, and towards embedding a significant amount of functionality within the C++ standard library itself. From a implementation-complexity standpoint, and when considering the necessary upgrade times and timescales for responding to security issues, this looks much less appealing.

§ 4.5. Should there be some integration with WebSockets?

Once we have networking support, it might be possible to specify some kind of stream object that can be used other networking facilities, some way to generate such a stream, and some way to get the name of the stream for use in the web content. This seems like a reasonable extension to what is proposed here, and WebSockets are widely supported, although it does depart from the model of wrapping existing system services (as with the standardized HTTP-server question).

§ 4.6. Should there be some use of executors in this library?

Yes. Once we have executors, the interface proposed here should use them to specify where the URI-scheme handlers are run.

§ 4.7. What happens when the `web_view` object is destroyed?

In the current proposal, the destructor blocks until any callbacks have finished executing, and then it closes the window. This is expected to be the easiest-to-use behavior (and aims to avoid undefined behavior). There was an explicit vote in SG1 in Cologne on this aspect of the interface:

Destructing a `web_view` that is not closed (with an `is_closed()` member) should be UB

SF F N A SA

1 1 6 4 0

§ 4.8. Isn't dealing with text (in HTML, JSON, URIs, etc.) ugly, verbose, difficult, error prone, or similar?

Interacting with HTML, etc. is cumbersome in C++ without additional libraries (especially as presented in the example in this paper). I fully expect that, if we decide to go down this route, additional utilities and abstractions will be added to go along with it. Also, even with only the presented interface, I can certainly imagine constructing a program with mostly-static HTML input combined with some kind of reactive JavaScript library (e.g., [Vue.js](https://vuejs.org/)) that's reasonably clean (at least in the sense that it does not involve the gratuitous composition of markup text in C++, although it might involve use of JavaScript injection to update values).

§ 4.9. Why don't we just standardize Qt's API?

Qt is not the only framework in this space, but it seems representative. From <http://doc.qt.io/qt-5/classes.html>, there are 1,594 classes in the Qt 5 API. From <http://doc.qt.io/qt-5/functions.html>, there are 12,984 functions. Dealing with such a large API seems impractical for our committee process, and frankly, this still seems impractical even if we were to strip this down significantly. Specifically, I don't really see the committee taking a hands-off approach to the content of the interface. Even if we could start with the text of Qt's documentation, we'd need to make sure that each class and function were specified in sufficient detail to allow for an independent (clean-room) implementation. I'm sure that suggestions would be made to change aspects of the interface to better expose capabilities of different vendors' underlying implementations, and future systems, and these would be discussed.

It has been further suggested that I look at QML/QtQuick, instead of QtWidgets, as a point of comparison. However, it's not clear why standardizing something which looks like QML/QtQuick would be more useful than using web content. QML also uses JavaScript. Also, the discussion of "Quick Controls 1 vs. Quick Controls 2" (<http://doc.qt.io/qt-5/qtquickcontrols2-differences.html>)

seems to be provide some non-trivial choices around desktop vs. mobile development. This could be a viable path for a standardization effort. It would, however, almost certainly invite design discussions around the API, the controls, etc. (because we can't hand that off to another standard, so we'd need to do that part ourselves), and we'd need a small-enough API surface for that to be practical.

§ 5. Proposed Interface

```
namespace std {
    template <typename T>
    concept URISchemeHandler = requires(T handler, const std::u8string &uri,
        { handler(uri, os) } -> std::error_code;
    };

    struct web_view {
        using native_handle_type = implementation-defined; // Defined as in 32.
        using native_options_type = implementation-defined; // Defined as in 32

        web_view(const std::u8string &title = "");
        web_view(const native_options_type &options, const std::u8string &title)

        void display_from_uri(const std::u8string &uri);

        template <typename Ret, typename... Args>
        std::future<Ret> invoke(const std::u8string &func_name, Args&&... args)

        template <typename URISchemeHandler>
        void add_uri_scheme_handler(const std::u8string &scheme, URISchemeHandler)

        void wait(bool closed = true);
        void wait(std::error_code &last_errc, bool closed = true);

        template <typename Rep, typename Period>
        bool wait_for(const std::chrono::duration<Rep, Period>& rel_time, bool
        template <typename Rep, typename Period>
        bool wait_for(const std::chrono::duration<Rep, Period>& rel_time, std::

        template<class Clock, class Duration>
        bool wait_until(const chrono::time_point<Clock, Duration>& abs_time, bo
        template<class Clock, class Duration>
        bool wait_until(const chrono::time_point<Clock, Duration>& abs_time, st

        void close();

        native_handle_type native_handle(); // Defined as in 32.2.3.
    };
}
```

An implementation of this interface is available from my github page: [web_view](#). This implementation uses the wxWidgets [wxWebView](#) class which implemented in terms of the platform-native APIs listed above for IE (on Windows), WebKit (on macOS/iOS), and WebKitGTK+ (on Linux and other platforms). Qt also provides a [QtWebView](#), but cannot currently fully support the proposed interface because it does not provide URI-scheme handlers. There are several others wrappers of these kinds that I found on github embedded inside of other projects (e.g., [FireBreath](#), [TwitchSwitcher](#), and [SumatraPDF](#) (this one is IE-only, but see references therein for interesting details)).

The proposed interface does not have a function to display a web page directly from a string (or similar). Such a facility could certainly be provided given the underlying interfaces provided by the browser APIs. However, if that page is going to contain links to further content that the application will provide, then a URI-scheme handler will be needed regardless. If the content is truly static and resides in a file, the a path to the file can be provided to the `display_from_uri` method. It should be noted that, as a limitation derived from current implementations, the URI scheme handlers provide a kind of virtual file-system interface, and as such, do not support POST data being provided to the handler along with the URI itself. To support that, and other protocols directly, we may need to provide an actual socket-based server to which the `web_view` could connect.

It is important to note that the handlers, both the URI-scheme handler, and the close handler, likely must be allowed to run in a different thread from the thread that created the `web_view` object. For security and other reasons, many implementations will separate the web-content rendering and script execution into a separate process. The `web_view` interface must allow for this, and in practice, this implies that the interface should avoid fine-grained interaction between the C++ application and the web content (as, in many implementations, all such interactions are mediated by remote procedure calls (RPC), and such calls are relatively expensive). Specifically, traversing and altering the DOM (document object model) of the content of the `web_view` from the C++ application, even if supported by the underlying platform APIs, would not only require a larger API surface, but would likely also be an expensive API to use.

§ 6. An Example

Here's an example application which uses the proposed interface. It is the test in the prototype implementation repository, and while not the simplest possible application, it makes use of all of the parts of the proposed interfaces, and moreover, makes it clear that asynchronous programming may be unavoidable in this space.

```

#include <web_view>

#include <vector>
#include <string>
#include <chrono>
#include <format>
using namespace std::chrono_literals;

int main(int argc, char *argv[]) {
    std::vector<std::string> args(argv, argv + argc);

    std::web_view w("web_view test app");
    w.add_uri_scheme_handler("wv", [&](const std::u8string &uri, std::ostream
        std::cout << "request: " << uri << "\n";
        os << std::format("<html><head><title>{0}</title><script>"
            "function newitem() {"
                "var node = document.createElement('li');"
                "node.appendChild(document.createTextNode('Time has"
                "document.getElementById('dl').appendChild(node); "
                "var d = new Date(); d.getTime();"
            "}")
            "</script></head><body><p>{0}</p><table>\n", uri);
        for (auto &a : args)
            os << std::format("<tr><td>{></td></tr>\n", a); // we need some kind
        os << "</table>";
        os << std::format("<p><a href=\"{>/more.html\">more</a></p>", uri);
        os << "<ul id='dl'></ul>";
        os << "</body></html>";

        return true;
    });

    w.display_from_uri("wv://first.html");

    std::error_code e;
    w.wait(false, e);
    std::cout << "initial display complete: " << e << "\n";

    while (!w.wait_for(2000ms)) {
        auto r = w.invoke<double>("newitem");
        std::cout << "got from script: " << r.get() << "\n";
    }
}

```

```
std::cout << "web_view closed\n";

return 0;
}
```

§ 7. Very-Preliminary Wording

Header `<web_view>` synopsis [**web_view.syn**]:

The header defines a class for providing a web-content-driven interface for the purpose of external user interaction.

```
namespace std {
    template <typename T>
    concept URISchemeHandler = requires(T handler, const std::u8string &ur
        { handler(uri, os) } -> std::error_code;
    };

    class web_view;
}
```

class `web_view` [**web_view**]

```
class web_view {
    ... // see above
};
```

Each `web_view` class instance represents an independent, asynchronous web-content interface. The provided `web_view` shall support content complying with the [\[HTML5\]](#), [\[PNG\]](#), and [\[ECMAScript\]](#) standards.

[*Note*:

Implementations are encouraged to support the latest [WHATWG living standards](#), [\[wai-aria\]](#), [\[WebGL\]](#), [\[webrtc\]](#), [\[webvtt1\]](#), [\[SVG11\]](#), CSS standards, DOM standards, and otherwise maximize

compatibility with other implementations (see, e.g., [Can I use...](#))

-- end note]

All `std::u8string` objects used with `std::web_view` must contain well-formed UTF-8 data.

The destructor blocks until all URI-scheme handlers have returned. These handlers are allowed to call member functions on a `web_view` object, even if another thread has concurrently started execution of the destructor.

`web_view` member functions [**`web_view.members`**]

```
web_view(const std::u8string &title = "");web_view(const native_options_
```

Effects: Constructs an object of the class `web_view` with the specified title.

[*Note:*

The title should be used by the implementation to associate a name with the web content's interface consistent with how that implementation generally displays the name of an interactive application. For example, on implementations that display graphical windows with title bars, the provided title, followed by a ": ", may be prepended to any title provided by the web content itself for the purpose of setting the text displayed in the title bar. The intent is that, from the user's perspective, the title sets the name of the application associated with the web content's interface.

-- end note]

If the implementation defines a `native_options_type`, then the corresponding overload shall be provided.

```
std::future<std::error_code> display_from_uri(const std::u8string &uri);
```

Effects: Causes the top-level web content to be replaced with content loaded from the provided URI. The implementation shall support the URI format specified in [\[rfc3986\]](#).

Returns: A future containing the error code describing the final status of the request. It is implementation defined at what point during the content-loading process the error status is determined.

If resources are unavailable to display the requested content, then an error should be set in the returned future. The implementation shall not wait indefinitely for necessary resources to become available before setting the final error status.

[Note:

The implementation should handle this function as a top-level navigation request. All state associated with web content previously displayed should be rendered inaccessible to the content loaded as a result of calling this function. If the previous content caused additional windows to be opened, those windows should be closed.

-- end note]

```
template <typename Ret, typename... Args>std::future<Ret> invoke(const s
```

Effects: The implementation shall execute the named function in the context of currently-displayed web content. The named function may refer to a [\[ECMAScript\]](#) script function in the context of the current web content or any other function that might be made available in an implementation-defined manner. The implementation may define limits on the size of the provided arguments and the execution time of the script.

Returns: The result of the script shall be converted to the type Ret and set in the returned future. If the execution of the requested function, or any of the necessary type conversions are not possible, then the future's associated shared state will be abandoned.

The intent here is that some mapping between JSON entities and types will be specified, and some type-traits and/or reflection capability will be used to enable conversion between user-defined types and JSON entities. JSON serialization libraries are a common use cases for reflection schemes (see, e.g., [Boost Fusion Json Serializer](#), [Boost.Hana JSON example](#)). A similar mapping scheme must be used for the provided arguments.

The intent here is to support invoking ECMAScript functions while allowing for future enhancements (e.g., some kind of exported WebAssembly functions).

It is expected that, for security reasons, implementations might restrict or disable this interface for web content provided by some remote sources.

```
template <typename URISchemeHandler>void add_uri_scheme_handler(const st
```

Effects: Registers the provided callable object to handle the provided URI scheme. When requests are generated to URIs with the provided scheme, the function-call operator of the provided object is invoked. The handler is provided with the full URI requested and a reference to a `std::ostream` into which the requested data should be stored. If an error is encountered, an appropriate error code should be returned. Methods on the provided handler object may be simultaneously called on threads other than the thread calling this function. Such calls need not be synchronized with each other.

```
template <typename Rep, typename Period>bool wait_for(const std::chr
template <typename Rep, typename Period>
bool wait_for(const std::chrono::duration<Rep, Period>& rel_time, std::e
```

Effects: Waits until the web content's interface becomes irrevocably unusable, or if closed is false, until the top-level web-content loading is complete, or the specified time interval has elapsed. If the return value is true, then `last_errc` shall be set.

Returns: `true` if web content's interface has become irrevocably unusable, or if closed is false, if the top-level web-content loading is complete.

Throws: Timeout-related exceptions (32.2.4).

```
template<class Clock, class Duration>bool wait_until(const chrono::t
template<class Clock, class Duration>
bool wait_until(const chrono::time_point<Clock, Duration>& abs_time, std
```

Effects: Waits until the web content's interface becomes irrevocably unusable, or if closed is false, until the top-level web-content loading is complete, or the specified time has been reached. If the return value is true, then `last_errc` shall be set.

Returns: `true` if web content's interface has become irrevocably unusable, or if closed is false, if the top-level web-content loading is complete.

Throws: Timeout-related exceptions (32.2.4).

```
void wait(bool closed = true); void wait(std::error_code &last_errc, bool
```

Effects: Waits until the web content's interface becomes irrevocably unusable, or if closed is false, until the top-level web-content loading is complete.

Throws: Timeout-related exceptions (32.2.4).

Concurrent invocation of wait is allowed.

```
void close();
```

Effects: Requests, asynchronously, that the web content's interface become irrevocably unusable.

§ 8. Acknowledgments

I'd like to thank JF Bastien, Botond Ballo, Jeffrey Yasskin, Mike Spertus, Bryce Lebach, Vinnie Falco, and Chandler Carruth for feedback and suggestions.

Some public discussion is available on [reddit](#). Also, see this thread on [Gecko's list](#).

The telecon SG16 review: [Notes](#)

The Argonne Leadership Computing Facility is a DOE Office of Science User Facility supported under Contract DE-AC02-06CH11357.

§ References

§ Informative References

[ECMAScript]

[ECMAScript Language Specification](#). URL: <https://tc39.github.io/ecma262/>

[HTML5]

Ian Hickson; et al. [HTML5](#), 27 March 2018. REC. URL: <https://www.w3.org/TR/html5/>

[P0267R7]

Michael B. McLaughlin, Herb Sutter, Jason Zink, Guy Davidson. [A Proposal to Add 2D Graphics Rendering and Display to C++](#). 10 February 2018. URL: <https://wg21.link/p0267r7>

[P0939R0]

B. Dawes, H. Hinnant, B. Stroustrup, D. Vandevorode, M. Wong. [Direction for ISO C++](#). 10 February 2018. URL: <https://wg21.link/p0939r0>

[P1062R0]

Bryce Adelstein Leibach, Olivier Giroux, Zach Laine, Corentin Jabot, Vittorio Romeo. [Diet Graphics](#). 7 May 2018. URL: <https://wg21.link/p1062r0>

[PNG]

Tom Lane. [Portable Network Graphics \(PNG\) Specification \(Second Edition\)](#). 10 November 2003. REC. URL: <https://www.w3.org/TR/PNG/>

[RFC3986]

T. Berners-Lee; R. Fielding; L. Masinter. [Uniform Resource Identifier \(URI\): Generic Syntax](#). January 2005. Internet Standard. URL: <https://tools.ietf.org/html/rfc3986>

[RFC6455]

I. Fette; A. Melnikov. [The WebSocket Protocol](#). December 2011. Proposed Standard. URL: <https://tools.ietf.org/html/rfc6455>

[SVG11]

Erik Dahlström; et al. [Scalable Vector Graphics \(SVG\) 1.1 \(Second Edition\)](#). 16 August 2011. REC. URL: <https://www.w3.org/TR/SVG11/>

[WAI-ARIA]

James Craig; Michael Cooper; et al. [Accessible Rich Internet Applications \(WAI-ARIA\) 1.0](#). 20 March 2014. REC. URL: <https://www.w3.org/TR/wai-aria/>

[WebGL]

Dean Jackson; Jeff Gilbert. [WebGL 2.0 Specification](#). 12 August 2017. URL: <https://www.khronos.org/registry/webgl/specs/latest/2.0/>

[WEBRTC]

Adam Bergkvist; et al. [WebRTC 1.0: Real-time Communication Between Browsers](#). 27 September 2018. CR. URL: <https://www.w3.org/TR/webrtc/>

[WEBVTT1]

Silvia Pfeiffer. [WebVTT: The Web Video Text Tracks Format](#). 10 May 2018. CR. URL: <https://www.w3.org/TR/webvtt1/>